

크로스아키텍처 실험보고서

Part III 의 모든 HATIR 실험을 8 개 가속기에서 동일하게

개요. study paper Part III 의 twin-의존 HATIR 실험 전부 (E1.1 decode-baseline · E1.2 state-placement · E1.3 kv-vs-ttt · E1.4 frequency-tiers · E1.5 kvmgr-gap · E3 analytical · E4 scaling) 를 8 개 아키텍처에서 동일하게 재실행하고, 실험별 그래프 · tier 배치 · 종합 한눈에 뷰 · 환경 · 고찰 · insight 를 정리한다. (E2.1/E2.2 는 host-CPU roofline 모양 측정이라 아키텍처 무관 — § 4.8.)

핵심 — 정직성 계약. 최우선 가치는 성능 수치 신뢰성이다. 각 twin 은 필드별 provenance+confidence(GOLD/SILVER/BRONZE) 로 저작하고 2라운드적대 검증을 통과시켰다. 단정 가능한 것은 GOLD/SILVER load-bearing 필드에 기반한 비율 · crossover-bound-tier 순서뿐이고, BRONZE(Vera Rubin 전채) 와 모든 절대 $\mu\text{s}/\text{mJ}$ 은 ideal roofline 하한 (directional/예비) 이다. 근거: AUTHORING-SPEC.md, specs/*.json, REVIEW-LOG.md, 본서 부록 E.

1. 방법

1. **twin 저작 (라운드 1, 28 agents).** 7 개 신규 칩을 다중 독립 검색 → 저작 → 독립 검증가 2 명 (vendor / 3rd-party) 이 신선 검색으로 반박 → 조정. 필드별 provenance+confidence.
2. **결과 재검증 (라운드 2, 10 agents).** 8 개 headline 독립 재확인 + 결과표 산술 · 물리 감사 + 과대주장 감사. (라운드 2 가 wse3 sparse → dense peak 오류를 잡음 — § 6.)
3. **실험 driver.** run_all_experiments.py 가 7 개 실험 전부를 임의 twin 으로 파라미터화 (HBM GPU + SRAM-only 공통). 모든 공식은 experiments/{E1.1,E1.2,E1.3,E1.4,E3} verbatim. **정확성 게이트** (--validate): H100 twin 이 확정 anchor 5 종을 <0.02% 재현 — rmw 6.442 GB/tok, ms 1.923, state 134.2 MB, S^* 65536, $B_{\max}$ 5.2.
4. **스키마 검증.** 8 개 twin 전부 hatir validate_twin PASS(error 0). 그림: make_figures.py → figures/*.png.

anchor: neural-mem-1.3B ($d=2048$, $m=16$, $L=24$, GQA-8, bf16).

1.1 실험의 모델 가정 — 무엇을 test-time 갱신 (decode RMW) 으로 세는가

decode 비용을 지배하는 것은 “매 토큰 test-time 갱신되는 state” 의 RMW traffic 이다. 그 state 를 논문별 실제 구조로 분해한다 (단일 $m=16$ 뭉뚱그림 → 컴포넌트+optimizer 분해). 단위 = $\text{params}/\text{layer} \div d^2$. 구성요소: $W_{\text{qkv}}(3d^2, Q \cdot K \cdot V \text{ 투영}) \cdot \text{nm}(8d^2, 2\text{-layer neural memory MLP}) \cdot Q\text{-K projection } \Pi(d^2, \text{TNT}) \cdot \text{cms_fast}(8d^2, \text{빠른 CMS 레벨}) \cdot \text{gate } \eta, \alpha (\approx 0)$.

모델	decode 시 RMW 되는 것	state/d ²	근거
titans-m16 (anchor)	nm + momentum	16	Titans; 게이트 baseline
TNT	local nm + Q-K projection Π (per token) — global 은 prefill 이라 제외	18	결정 =local(chunk-1), prefill=global, Π 읽기마다
HOPE-DGD	Wqkv + nm + cms_fast 전부 self-modifying × DGD(1.0)	19	self-mod Titans + CMS fast
HOPE-DeltaMom	" × Delta Momentum(2.0)	38	+momentum 버퍼
HOPE-M3	" × M3=Multi-scale Momentum Muon(3.5)	66.5	+fast/slow momentum + NS
Sleep (wake decode)	= HOPE attention side	19 – 66.5	slow CMS·expert 는 offline (§ 5)

HOPE optimizer 3 종 (DGD / Delta Momentum / M3) 이 momentum·preconditioner 버퍼 수를 정해 decode state 를 **3.5×** 좌우한다. **HOPE** = **attention** 처리 (Wq,k,v·gate·nm 전부 매 토큰갱신 — 님 지적대로 다 포함), **Sleep** = **FFN** 처리 (low-rank expert 를 offline consolidation 으로 증설; § 5). **Sleep expert** = **low-rank** {**A** $\mathbf{d} \times \mathbf{r}$, **B** $\mathbf{r} \times \mathbf{d}$ } = **2dr** (FFN-size 아님). E1.2 – E1.4 는 anchor(titans-m16) 를쓰고, **E1.1** 은 위 5 개 모델 전부를 켜다 (§ 4.1). 대상 칩 **vr200**(Vera Rubin NVL144; R100 은 초기 die 명) — BRONZE 이라결론 무관.

2. 대상 8 종 + 신뢰도등급

twin	칩	tier	backing
h100	NVIDIA H100 SXM5	GOLD	HBM3
mtia2	Meta MTIA v2	GOLD	LPDDR5
b100	NVIDIA B100 (Blackwell)	SILVER	HBM3e
mi355x	AMD MI355X (CDNA4)	SILVER	HBM3e
tpu-v7	Google TPU v7 Ironwood	SILVER ※	HBM3e
groq-lpu	Groq LPU/TSP	SILVER	SRAM
wse3	Cerebras WSE-3	SILVER	SRAM(wafer)
vr200	NVIDIA Vera Rubin	BRONZE	HBM4

fig7 · 한눈에 — 크로스아키텍처 pair thesis (BW 오름차순; 색=신뢰도 등급)

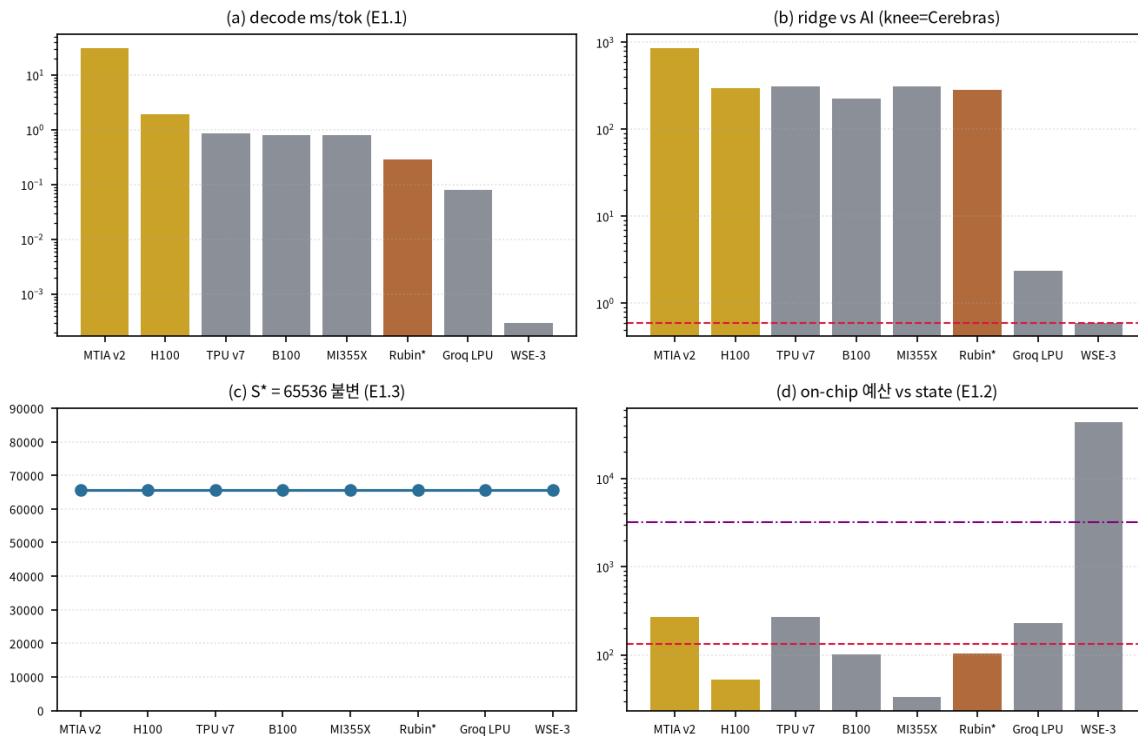


fig7 · 한눈에: (a) decode 비용 (b) ridge vs AI (c) S^* 불변 (d) on-chip 예산 vs state. BW 오름차순, 색=신뢰도 등급.

주의 vr200 은 내부 전면 미공개 → 전체 directional, 발표 단정 금지. tpu-v7(※) 은 backing/compute 는 GOLD 이나 on-chip VMEM(reuse-tier) 이미공개 추정 → residency(E1.2) 주장만 BRONZE. groq/wse3 는 HBM 이 없는 SRAM 기반.

3. 종합 — 한눈에

HW 스펙 (단일 가속기).

twin	tier	backing	BW (TB/s)	dense BF16/FP16 (PF)	용량	on-chip
h100	GOLD	HBM3	3.35	0.99	80 GB	50 MB
b100	SILVER	HBM3e	8.0	1.8	192 GB	101 MB
mi355x	SILVER	HBM3e	8.0	2.52	288 GB	34 MB
tpu-v7	SILVER	HBM3e	7.38	2.31	192 GB	268 MB ※
vr200	BRONZE	HBM4	22.0	6.25	288 GB	105 MB
mtia2	GOLD	LPDDR5	0.205	0.177	128 GB	268 MB
groq-lpu	SILVER	SRAM	80.0	0.188	220 MB	(backing)
wse3	SILVER	SRAM	21000	12.5	44 GB	(backing)

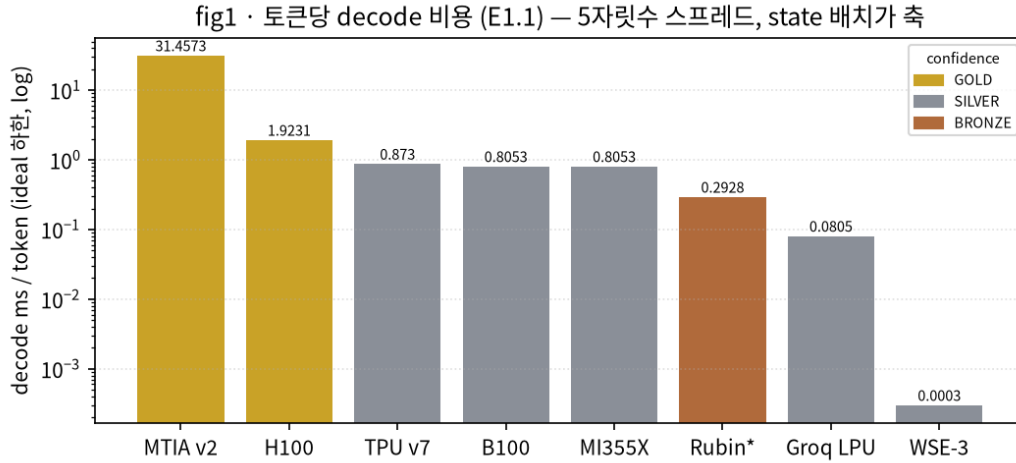


fig1 · 토큰당 decode 비용 (log). 5 자릿수 스프레드, state 배치가 축.

pair thesis 지표 (anchor).

twin	decode ms/tok	ridge	S*	C*	Bmax@10ms	bound (여유)
h100	1.923	295	65536	337	5.2	memory (497×)
b100	0.805	225	65536	249	12.4	memory (379×)
mi355x	0.805	315	65536	362	12.4	memory (530×)
tpu-v7	0.873	313	65536	359	11.5	memory (527×)
vr200	0.293 ※	284	65536	323	34.2 ※	memory (478×)
mtia2	31.457	864	65536	1180	0.32	memory (1455×)
groq-lpu	0.081 †	2.35	65536	2.4	124	memory (4×)
wse3	0.0003	0.60	65536	0.6	32596	knee (1.01×)

※ vr200/tpu-v7(residency) = BRONZE. † groq BW = vendor aggregate. 절대치는 ideal 하한 (단정 안 함).

4. 실험별결과 · 환경 · 고찰

4.1 E1.1 — decode-baseline (토큰당 whole-state RMW)

설명. decode 한 스텝이 fast-weight state 전체를 read-modify-write 하는 비용 (traffic·time·energy·bound). anchor 상태 = $m \cdot d^2 \cdot L = 134 \text{ MB/layer} \times 24$.

환경. 각 twin 의 backing tier(bandwidth_bps·energy_pj_per_byte) + compute leaf(peak_macs). traffic = $2 \cdot \text{state} \cdot L(\text{read} + \text{write})$, time = traffic/BW, energy = traffic·epb.

결과. decode ms/tok: MTIA 31.5 → H100 1.92 → Rubin 0.29 → Groq 0.08 → Cerebras 0.0003 (약 5 자릿수 스프레드). AI = **0.594** 로 8 개 전부 동일 (workload 고정). 에너지: SRAM 칩 (Groq/WSE-3) 6.4 mJ/tok, HBM/LPDDR 32 – 52 mJ/tok(epb 차이).

고찰 · insight. decode 가 memory-bound 라는 성질은 **update rule** 의 RMW 대칭성에서 오는 것이라 모델 · HW 와 무관하다 (AI 항상 0.594). 그러나 얼마나 깊이 memory-bound 인지 (여유 $1\times - 1455\times$) 와 절대 비용은 순전히 HW 의 BW·epb 문제다. **SRAM 기반 칩의 에너지 우위 ($7\times$)** 는 write-heavy 부하에서 특히 큰데, epb 가 HBM 의 1/7 이기 때문이다 — memory-centric 부하일수록 low-epb 메모리의 가치가 커진다.

E1.1b — 논문별 모델 decode (§ 1.1 구조 반영). anchor(m=16) 가 아니라 각 논문의 실제 갱신 구조로 켄 decode ms/tok:

모델	state/d ²	GB/tok	H100	MTIA v2	Cerebras
titans-m16 (anchor)	16	6.44	1.92	31.5	0.0003
TNT (local+Q-K Π)	18	7.25	2.16	35.4	0.0003
HOPE-DGD	19	7.65	2.28	37.4	0.0004
HOPE- DeltaMom	38	15.3	4.57	74.7	0.0007
HOPE-M3	66.5	26.8	7.99	130.7	0.0013

insight. 실제 HOPE 는 anchor 보다 훨씬 무겁다 — **HOPE-M3 는 titans-m16 대비 $4.2\times$ decode** ($W_q, k, v \cdot nm \cdot cms_fast$ 를 전부 multi-scale momentum 으로 갱신 하니까). **optimizer 선택만으로 DGD ↔ M3 가 $3.5\times$ 차이** — decode 예산이 뽀뽀하면 DGD(2.28ms), 표현력 필요하면 M3(7.99ms). TNT 는 Q-K projection Π (d²) 한 장 추가 라 anchor 와 근사 (18d²). 이순서 (titans<TNT<HOPE-DGD ≪ HOPE-M3) 는 **모든 HW 에서 동일 비율로** 유지된다 (HW 는 BW 로 세로 스케일만).

4.2 E1.2 — state-placement (tier 잔류)

설명. state 가 context-무관 고정 크기라 on-chip 상주가 designable 선택이다 (context 로 자라는 KV 와 대조). 각 HW 의 자기계층에서 backing vs on-chip RMW 비용 · 잔류 · residency crossover $d^*(1\text{-layer state 가 on-chip 에 들어가는 최대 } d)$ 를 켄다.

환경. 각 twin 의 on-chip reuse tier(L2/Infinity Cache/LDS/SRAM) 용량 · BW·epb vs backing. $d^* = \sqrt{(ondie_cap / 2m)}$.

결과.

twin	on-chip	판정	d*	on-die vs backing (E/T)
wse3	44 GB	whole-model 상주	37081	$1\times/1\times$ (backing=on-chip)
tpu-v7	268 MB ※	per-layer 상주	2896	$25\times/22\times$
mtia2	268 MB	per-layer 상주	2896	$20\times/13\times$
groq-lpu	220 MB	per-layer 상주	2685	$1\times/1\times$ (backing=on-chip)

twin	on-chip	판정	d*	on-die vs backing (E/T)
vr200	105 MB	backing 스트리밍	1810	—
b100	101 MB	backing 스트리밍	1774	12.5×/1.25×
h100	50 MB	backing 스트리밍	1280	17.5×/3×
mi355x	34 MB	backing 스트리밍	1024	12.5×/1.25×

고찰 ·insight. decode-state 병목은 본질적으로 **on-chip 용량 문제**다. HBM GPU(H100/B100/MI355X/Rubin)는 1-layer state(134 MB)조차 on-chip(L2 34–105 MB)에 못 담아 매 토큰 HBM 스트리밍한다 —그런데 만약 담을 수 있다면 에너지 12–17×, 시간 3× 이득이 걸려있다 (빈 기회). MTIA/TPU는 256–268 MB SRAM으로 layer 상주가 되고, **Cerebras는 44 GB로 전체 모델 상주** — 이 부하에서 유일하게 배치문제가 사라진다. 업계 추세 (HBM 용량↑)는 이 부하엔 덜 효과적이고, **on-chip SRAM ↑가 직접 답**이다. (※ tpu-v7의 on-chip 예산은 미공개 추정 = BRONZE, residency 판정은 예비.)

4.3 E1.3 — kv-vs-ttt (S* crossover, B_max)

설명. KV cache read(context S에 비례)와 TTT state RMW(상수)가 같아지는 crossover S*, 그리고 per-sequence state로 인한 동시 시퀀스상한 B_max.

환경. $S^* = m \cdot d^2 \cdot (\text{dtype 비}) / \text{kv_dim}$. $B_{\text{max}} = \text{BW} \cdot t_{\text{target}} / \text{rmw_per_token}$.

결과. $S^* = 65536 \text{ token}$, 8개 전부 동일. $B_{\text{max}}@10\text{ms}$: MTIA 0.32 → H100 5.2 → Cerebras 32596 (BW 비례). KV write%=0.003, TTT write%=100(대칭 RMW).

고찰 ·insight. S*는 이 스터디의 **가장 강한 결과**다 —crossover 위치가 backing 종류 (HBM/LPDDR/wafer SRAM)·BW 값·이용률어디에도 movable하지 않다 (순수 traffic 대수). 이것이 “상쇄 논증”의 직접실증이고, **BRONZE(Rubin) twin을 포함해 8개 전부에서 단정 가능한 유일한 절대적 결과**다. 반면 B_max는 순수 BW-bound(용량 아님) — decode는 용량 벽이 아니라 **대역폭 벽**에 먼저 막힌다.

4.4 E1.4 — frequency-tiers (cadence → memory tier)

설명. NL(HOPE)/Sleep은 memory level마다 update cadence를 준다. cadence가 memory 계층 배정으로 번역되는지 — per-token은 hot tier를 요구하고, sub-per-token은 amortize되어 cold tier로 내려갈 수 있는지.

환경. 각 twin의 on-chip·backing을 hot tier로, 일반 DDR5·CXL을 cold tier로. cadence C의 amortized 기여 = $(2S/\text{BW})/C$, per-token budget 100 μs 하 admissible 중 가장 cold tier 선택.

결과 (cadence → tier).

twin	every 1 tok	every 16+ tok
H100/B100/MI355X/TPU/Rubin	HBM	CXL
MTIA v2	on-die SRAM	CXL

twin	every 1 tok	every 16+ tok
Groq / Cerebras	SRAM	CXL

고찰·insight. per-token fast level 은 hot tier 에 고정되지만 그 **hot tier 가 HW 마다 다르다** — GPU 는 HBM, MTIA/Groq/Cerebras 는 on-chip SRAM. 즉 SRAM 기반 칩에선 매 토큰 갱신되는 fast weight 가 **native 로 on-chip 에 산다**. 그리고 sub-per-token(cadence ≥ 16) 은 어디서든 CXL 로 내려간다 — amortization 이 강력해서 NL 의 저주파 CMS level·Sleep 의 offline consolidation 은 값싼 pooled 메모리로 밀 수 있다. **NL/Sleep 의 다속도 메모리가 실제 메모리 계층에 어떻게 앉는지가 HW-상대적**이라는 것이 핵심 — 같은 알고리즘이 GPU 와 wafer-scale 에서 다른 배치 전략을 요구한다.

4.5 E1.5 — kvmgr-gap (KV-manager 범주불일치)

설명. KV cache manager 의 의미론 (content-addressed reuse, append-only, free-drop eviction) 이 RMW state 에 안 맞음을 보인다. 정량조각: manager 가 0 으로 값매기는 dirty writeback 이 실제로는 full write-back 을 빙긴다.

결과. dirty-writeback owed: Cerebras 134 μJ / MTIA 1342 μJ (KV-manager prices 0). reuse=0(content 매 토큰 변이), stale accretion(append-only 시 T 버전 누적).

고찰·insight. SW argument 의 뼈대 (reuse=0, stale 누적) 는 HW 무관이지만, **un-priced eviction 비용은 epb 에 비례해 high-energy backing(LPDDR MTIA)에서 가장 크다**. TTT-state manager 는 KV manager 에 없는 event 가 필요하다: update_in_place, mark_dirty/writeback, checkpoint/rollback, bind_to_sequence. 즉 서빙 SW 스택도 새로 써야 하는 부하다.

4.6 E3 — analytical (ridge, C*)

설명. 닫힌 형태 cost model 의 roofline knee(ridge = peak/BW) 와 chunk crossover C*.

결과. ridge: Cerebras 0.60 $\rightarrow$ Groq 2.35 $\rightarrow$ B100 225 $\rightarrow$ MTIA 864. C*(prefill compute-bound 되는 chunk): Cerebras 0.6 $\rightarrow$ H100 337 $\rightarrow$ MTIA 1180.

고찰·insight. ridge 는 각 HW 를 이 부하의 memory/compute 스펙트럼 위에 놓는 단일 숫자다. decode AI 0.594 는 6 종 HBM/LPDDR 의 ridge 보다 2–3 자릿수 낮아 압도적 memory-bound, Groq 는 $\sim 4\times$, **Cerebras 만 ridge $\approx$ AI(knee)**. C* 는 반대로 prefill/training 이 compute-bound 되는 chunk 크기 — MTIA 는 1180 이라 큰 chunk 가 필요하고, Cerebras 는 0.6 이라 chunk-1 조차 compute-bound 근처다. **한 알고리즘이 HW 에 따라 memory 문제도 compute 문제도된다.**

4.7 E4 — scaling (170M $\rightarrow$ 70B)

설명. state·traffic·S* 가 모델 규모 (d) 에 따라 어떻게 자라는지, 8 개 HW 각각에서.

결과. $\text{rmw} \propto m \cdot d^2 \cdot L$ (170M 0.45 $\rightarrow$ 70B 343 GB/tok). log-log 에서 **모든 HW 곡선이 평행** (동일 기울기), 세로 오프셋만 1/BW.

고찰·insight. 스케일링 거동은 **보편적**이다 — HW 는 곡선을 위아래로 옮길 뿐 형태를 안 바꾼다. 따라서 규모에 대한 결론 (state 는 width^2 로 자란다, S* 는 width^2 로 자란다)

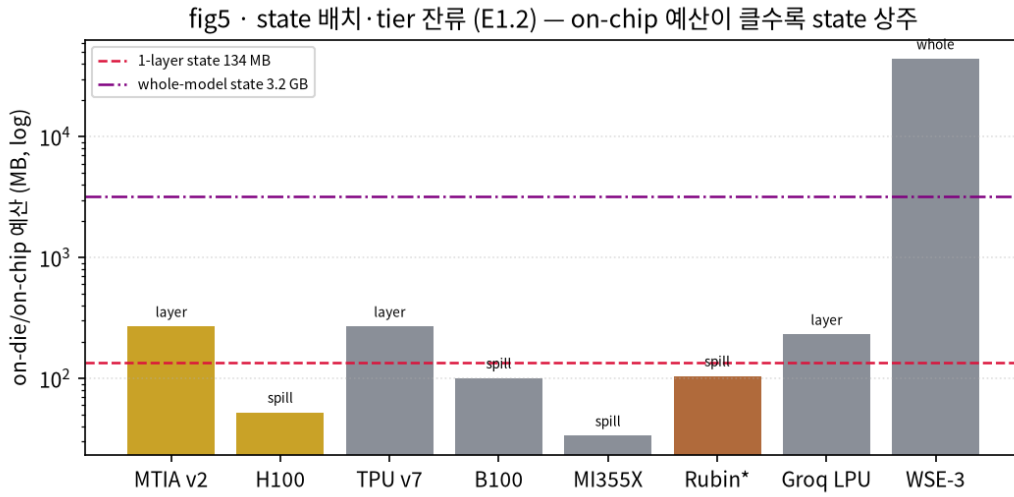


fig5 · state 배치 · tier 잔류 (log). on-chip 예산 vs 1-layer(134MB)·whole-model(3.2GB) state.

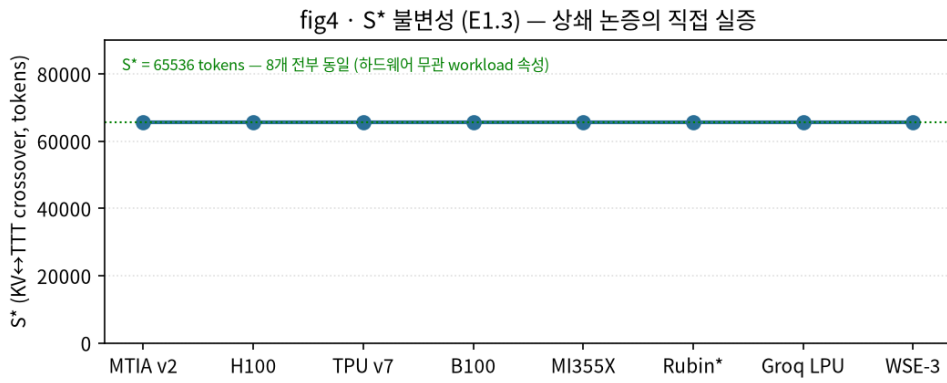


fig4 · S* 불변성 — 8 개 전부 65536. 하드웨어 무관 workload 속성.

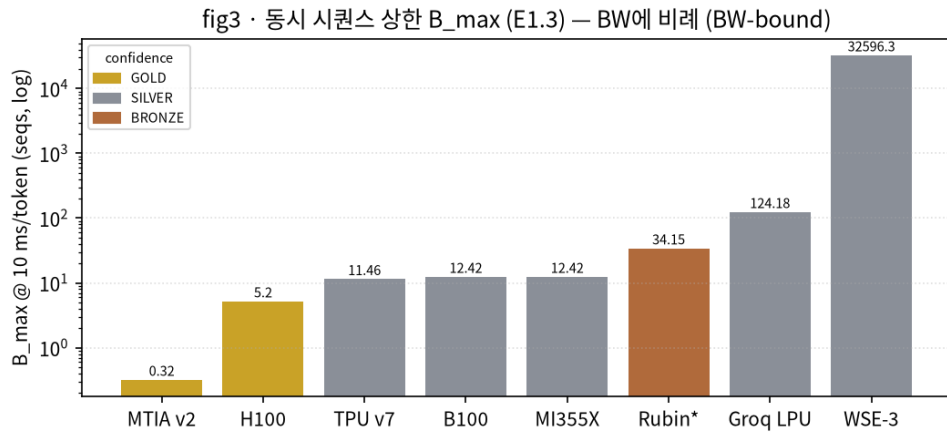


fig3 · B_max@10ms(log) — BW 에 비례 (BW-bound).

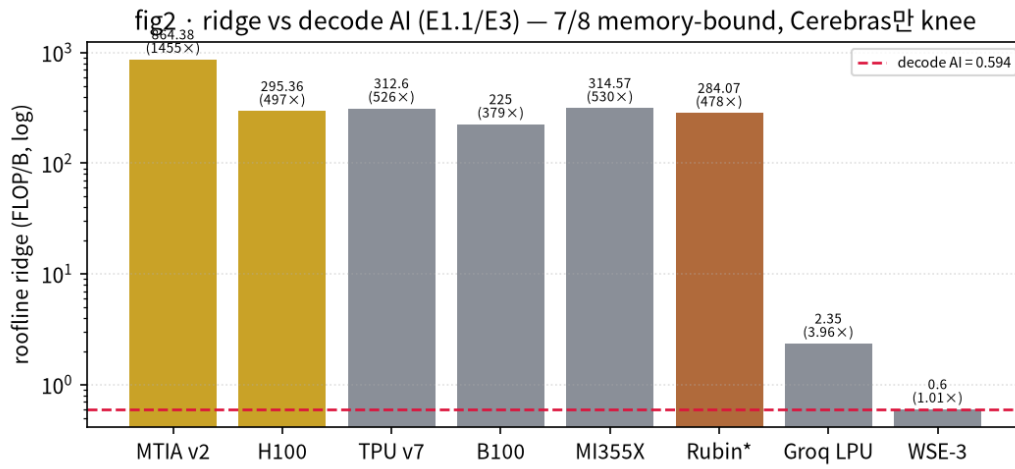


fig2 · ridge vs decode AI(log). 7/8 memory-bound(여유 380 – 1455×), Cerebras 만 knee.

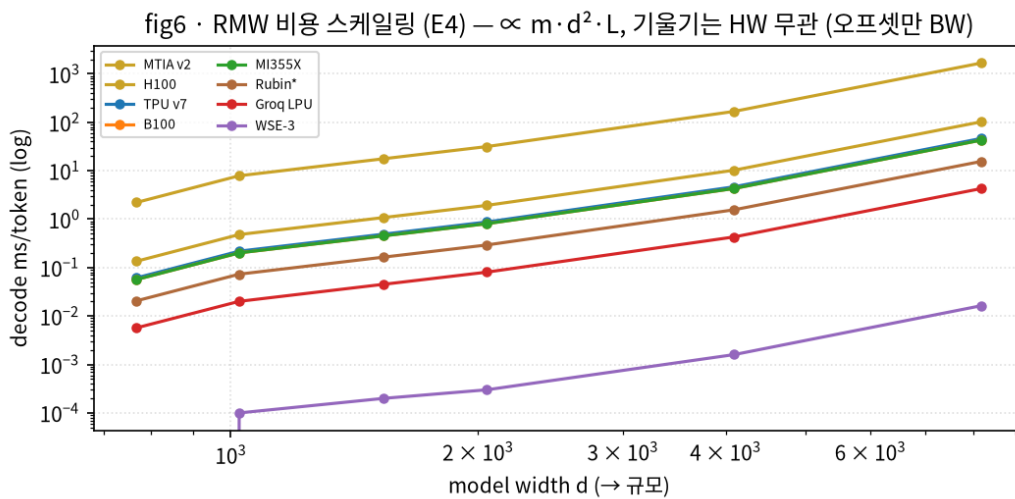


fig6 · RMW 비용 스케일링 (log-log). $\propto m \cdot d^2 \cdot L$, 기울기 HW 무관, 오프셋만 1/BW.

은 8 개 HW 전부에서 그대로전이된다. 큰 모델일수록 on-chip 상주가 불가능해지고 (state 가 GB 급) HBM 스트리밍이 강제되므로, § 4.2 의 “on-chip SRAM 이 답” 논지는 규모가 커질수록 강해진다.

4.8 E2.1 / E2.2 — host-CPU roofline (아키텍처무관)

E2.1(chunk-intensity)·E2.2(rmw-cliff) 는 이 **host CPU** 에서 실제시간을 켜 roofline 모양 측정이라 타깃 HW 별로 바뀌지 않는다 (host 는 host). 대신 각 twin 의 ridge (§ 4.6) 가 그 곡선의 knee 위치를 정한다 —host $C^* \approx 32$ vs H100 $C^* \approx 337$ 처럼 **모양은 이송되** **되 값은 ridge 의존**. 이 두 실험은 study paper 원본 그대로 유효하다.

5. HOPE / Sleep 적용 모델 심층 분석 — 1T 스케일링

앞 실험들은 추상 anchor(neural-mem-1.3B) 였다. 여기서는 실제 적용모델 — **HOPE(Nested Learning) 와 Sleep** — 의 구조를 그대로 모델링해 total 파라미터를 1T 까지 올리며 decode 비용을 켜다. (근거: notes/2512.24695, notes/2606.03979; 코드 hope_sleep_scaling.py.)

구조 (grounded). HOPE block = **self-modifying Titans**(6 개 test-time memory: $k, v, q, \eta, \alpha, +$ — 투영 자체가 갱신) + **CMS chain**($MLP^{(f_1..f_k)}$, fast level 은 매 토큰 RMW, slow level 은 chunk $C \times 1$ 마다). **Sleep** = wake/sleep lifecycle: wake=HOPE, **sleep(offline)** 에서 fast block 지식을 느린 block 에 **low-rank expert** 로 **append**(expert pool 이 sleep 마다 증가)하고 dreaming. **핵심: slow-level 갱신·expert 성장이 전부 offline** —decode critical path 밖.

모델링. decode 는 memory-bound per-token RMW traffic 이지배 (Part III). “매 토큰 RMW 되는 state” 를 세 regime 으로:

- dense-ttt — 전체 모델이 test-time-learned (strawman 상한): $RMW \propto P_{total}$.
- hope — self-mod + 모든 CMS level 이 inference 중갱신 (fast $C=1$ + slow $1/C$ amortized).
- sleep — self-mod + **fast level 만** (slow-level consolidation·expert 성장은 offline).

두 스케일링 recipe 를 비교한다: **WIDTH**(d, L 성장 — dense backbone) vs **EXPERT**(backbone 고정, total 을 Sleep-appended expert pool 로만, MoE top-2 routing).

모델은 § 1.1 의 보정 구조를 썸: 매 토큰 $RMW = HOPE \text{ attention side}(Wqkv + nm + cms_{fast}) \times optimizer$. $optimizer = M3(3.5, \text{보수적 headline})$. $expert = \text{low-rank} \{A d \times r, B r \times d\} = 2dr$, MoE top-2, **offline consolidation**.

결과 A — WIDTH (d, L 성장). attention-side state 가 $d^2 L$ 로 자라 **Sleep decode** 도 증가, 소규모에선 dense 와 거의 같음.

fig8 · HOPE/Sleep decode 비용 1T 스케일링 — Sleep은 expert 성장을 offline로 밀어 decode를 묶는다

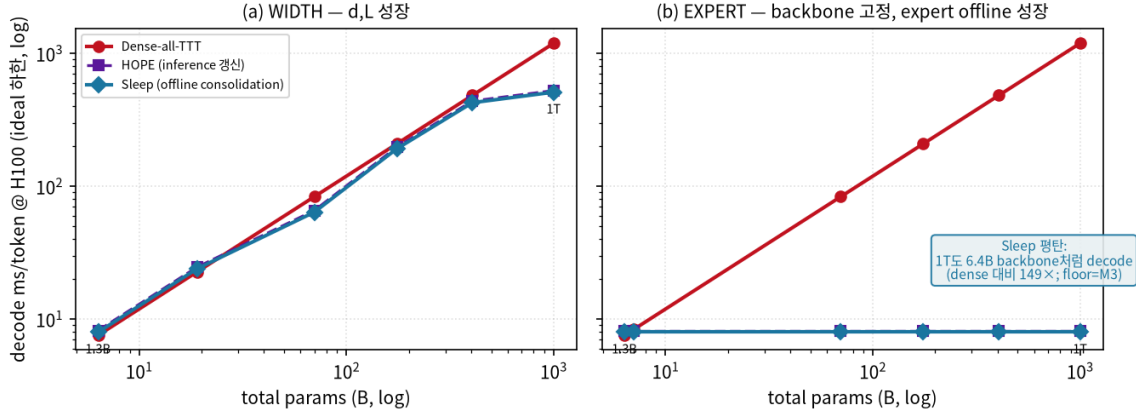


fig8 · HOPE/Sleep decode 1T 스케일링 (보정 모델: Wqkv+nm 갱신, low-rank expert). (a) WIDTH: Sleep 도 상승, 소규모선 dense 와 동급. (b) EXPERT: Sleep 평탄 — 1T 도 6.4B backbone 처럼 decode(149×).

total	active	S_sleep	sleep ms/tok ※	dense ms/tok	dense/sleep
6B	6.3B	13.4 GB	8.0	7.6	1×
70B	51B	107 GB	63.9	83.5	1×
405B	339B	714 GB	426	484	1×
1T	407B	857 GB	512	1194	2×

결과 B — EXPERT (backbone 고정 d=2048,L=24, low-rank expert 만 offline 성장). active·S_sleep·decode 상수.

total	#exp(low-rank)	active	S_sleep	sleep ms/tok ※	dense ms/tok	dense/sleep
6B	0	6.4B	13.4 GB	7.99	7.6	1×
70B	2530	6.4B	13.4 GB	7.99	83.6	10×
405B	15841	6.4B	13.4 GB	7.99	484	60×
1T	39484	6.4B	13.4 GB	7.99	1194	149×

Sleep decode FLOOR(EXPERT family, 1.3B → 1T 평탄) — HOPE optimizer
별: DGD 2.28 ms (3.83 GB) · **DeltaMom 4.57 ms** (7.65 GB) · **M3 7.99 ms** (13.4 GB).
 optimizer 가 floor 를 3.5× 좌우.

※ H100, ideal roofline 하한. 절대치 단정 안 함 — **비율 · 평탄성**이 load-bearing.

고찰 · insight (님 가설 검증). 님의 직관 — “Sleep 은 scaling 가능” — 은 맞다. 보정 된 모델 (Wqkv+nm 전부 갱신, low-rank expert) 에서도 핵심이 유지된다:

1. **Sleep 의 offline consolidation 이 total 파라미터를 decode 에서분리한다.** EXPERT family 에서 total 6B → 1T(167×) 에도 **per-token RMW state 13.4 GB 상수, decode 7.99 ms 완전 평탄.** 39484 개 low-rank expert(1T 의 대부분) 가 **decode RMW 에 0 기여** — offline 갱신 + MoE top-2 routing. 1T Sleep 모델이

6.4B backbone 처럼 decode. 1T 에서 dense 대비 $149\times$. (보정 전 $903\times$ 는 과대였음 — 실제 attention-side floor 가 더 무겁다.)

2. “무엇을 키우느냐”가 관건. WIDTH family(d,L 성장)에선 Sleep 도커지고 소규모선 dense 와 동급 ($1\times$) — attention-side RMW 가 d^2L 이라. 즉 **self-mod·fast-level width** 를 키우면 **decode** 가 값을 치르고, FFN 을 **low-rank expert** 로 **offline** 증설하면 **decode** 무료. 1T 승리 recipe = **HOPE attention backbone(+optimizer 는 DGD 로 가볍게) 고정 + capacity 는 offline low-rank expert pool.**
3. **decode FLOOR** 는 **HOPE optimizer** 가 정한다. floor 2.28(DGD) – 7.99(M3) ms. 표현력이 필요 없으면 DGD 로 floor 를 $3.5\times$ 낮춘다. **Sleep 은 성장 (expert) 을 offline 로, HOPE 는 표현력 (optimizer) 을 decode 예산으로** — 두 손잡이가 분리된다.
4. 이것이 **Sleep** 의 진짜 systems 기여. Titans/HOPE 는 test-time 갱신을 decode 에 넣어 memory-bound RMW 를 만들었다 (Part III 부담). **Sleep 은 성장하는 memory(expert) 를 offline(수면) 으로 빼** continual-learning capacity 를 무한히 키우면서 serving decode 는 attention floor 로 묶는다 — **Part III decode-state 병목의알고리즘적 해답.**

정직한 한계. (a) decode 비용 모델이지 품질아님 — 논문 $\leq 1.3B$ 실측, 39484-expert 스케일은 저자 미보고·본 분석 투영. (b) 절대 ms 는 ideal 하한. (c) optimizer mult(DGD $1/\Delta$ Mom $2/M3$ 3.5)·rank 256·top-2 는 논문 구조 근거 파라미터화 (정확 폭 미공개). 그럼에도 **load-bearing 결론 (EXPERT family decode 평탄, expert 는 decode 무료, floor 는 optimizer 가 결정)** 은 구조에서 직접 따라와파라미터에 robust.

6. 종합 insight · 분석

1. **pair thesis** 의 **load-bearing** 결론은 하드웨어 공간 전체에서보편적이다. S^* 불변 (§ 4.3), decode memory-bound (§ 4.1/4.6, 7/8), tier 순서 (§ 4.2), 스케일링 형태 (§ 4.7) — 전부 HBM GPU·LPDDR NPU·wafer SRAM 을가로질러 성립. 이 스터디가 준 새 증거: 이 명제들은 특정 칩이 아니라 **update rule** 의 구조 (고정크기·RMW 대칭) 에 걸려 있다.
2. HW 가 바꾸는 유일한 것은 절대 비용이고, 그 축은 **on-chip 용량·대역폭 (state 배치)** 이다. decode-state 는 append-once KV 와 달리고정크기라 on-chip 상주가 designable 이고 (§ 4.2), 상주하면에너지·시간이 한 자릿수 개선된다. **SRAM-heavy 설계 (Cerebras 전체 상주 / Groq layer 상주) 가 이 memory-centric 병목을 구조적으로해소** — 이것이 본서 memory-centric 논증의 하드웨어 증거다.
3. **Cerebras knee** 는 경고이자 방향이다. 8 개 중 유일하게 decode 가 memory-bound 를 벗어나는데 (ridge $0.60 \approx AI$), wafer SRAM 의 극단적 BW 가이유다. 즉 “충분히 BW-rich 하면 이 부하도 balanced 가 된다” — 미래소자 방향의 힌트. (단 dense peak 교정 후의 결과 = 라운드 2 가 sparse 수치를 잡음.)

4. 정직성이 결론을 약화하지 않고 정확히 한다. 절대 $\mu\text{s}/\text{mJ}$ 는 ideal 하한이라 단정하지 않지만, 우리가 단정하는 양 (비율 $\cdot \text{crossover} \cdot \text{bound} \cdot \text{tier}$) 은 8 개 HW·전 규모에서 견고하다. 상쇄되는 양만 단정하고 상쇄 안 되는 양은 유보 — 이 절제가 8 칩실측으로 정당화됐다.

7. 신뢰성 — per-quantity + 리뷰감사

결과	의존	단정?
$S^* = 65536$ (전 아키텍처)	workload 만 (HW 무관)	예 — 전부 (BRONZE 포함)
bound = memory (7/8)	ridge $\gg 0.59$ (380 – 1455 $\times$)	예 — HBM/LPDDR/Groq
Cerebras = knee	ridge $0.60 \approx \text{AI } 0.59$	예 — “경계” 로
decode 비용 순서 (SRAM $\ll$ HBM $\ll$ LPDDR)	backing BW (GOLD/SILVER)	예 — vr200 제외
tier 잔류 / d^*	on-chip 용량	예 — tpu-v7 은 BRONZE
절대 $\text{ms}/\text{tok} \cdot \mu\text{J} \cdot \text{mJ}$	BW·epb + ideal 가정	유보 — ideal 하한
vr200 전체	미공개 스펙	유보 — directional

라운드 2 가 잡아 고친 것 (다중 검증이 작동한 증거). [MAJOR] wse3 peak 125 PF(sparse 마케팅) $\rightarrow$ 12.5 PF(dense): ridge 5.95 $\rightarrow$ 0.60, Cerebras knee 발견. [MINOR] B_max 전사오류 수정, robustness 주장 HBM/LPDDR 로 범위한정, 절대치 vr200 제외. 상세: REVIEW-LOG.md. HATIR·HAT spec 자체의 신뢰성 (ZigZag byte-exact 검증, ideal-vs-실측 gap): 본서 부록 E.

8. 재현

```
cd ~/repos/neural-memory-study ; PY=.venv/bin/python
$PY multiarch/run_all_experiments.py --validate # H100==확 정 anchor 게이트
$PY multiarch/run_all_experiments.py --all # 8 twin x 7 실험 -> results_full/*.json
$PY multiarch/hope_sleep_scaling.py # HOPE/Sleep 1T 스케일링 (§5)
$PY multiarch/make_figures.py # figures/fig1-7
$PY multiarch/make_scaling_fig.py # figures/fig8
```

twin: multiarch/twins/*.json (validate_twin PASS). 출처 · 등급: multiarch/specs/*.json. 실험 상세: results_full/*.json. driver 는 E3/E1.x 공식 verbatim — H100 재현 게이트가 충실성 보증.